Study Guide — IITG-AIML-2503 Offline Exam

Contents

1. PyTorch & Neural Network Foundations 1
2. Convolutional Neural Networks (CNNs) 3
3. Autoencoders & Generative Models 6
4. Object Detection & Image Segmentation 7
5. Recurrent Neural Networks: RNN, LSTM, GRU 10
6. NLP Foundations & Word Embeddings 12
7. Attention Mechanism & Transformer Architecture 14
8. BERT, GPT, ELMo, GloVe & the LLM Landscape 17
9. Building & Fine-Tuning Transformers in Practice (Hugging Face) 19
10. Retrieval-Augmented Generation (RAG) Systems 21
11. Other Topics Briefly Introduced (context, not deeply taught) 24

Batch: iitg-aiml-2503 · **Program:** Credit Linked Program in AI & ML

This guide compiles the key concepts you need for your upcoming evaluation, covering the full breadth of the course: PyTorch foundations, CNNs, generative models, object detection, sequence models (RNN/LSTM/GRU), NLP and word embeddings, attention and Transformers, BERT/GPT and the LLM landscape, Graph Neural Networks, and Retrieval-Augmented Generation (RAG) systems. Work through each section, focus on the formulas and worked examples, and use the "Key terms" line at the end of each section to self-test.

1. PyTorch & Neural Network Foundations

1.1 Tensors and device management

- A **tensor** is the fundamental data structure in deep learning: scalars (0D), vectors (1D), matrices (2D), higher-order tensors. PyTorch tensors differ from NumPy arrays in two ways: GPU acceleration and automatic differentiation (tracking computation history).
- Three properties that matter: **shape** (dictates matrix-operation compatibility; mismatches cause runtime errors), **dtype** (floating point, usually 32-bit, needed because gradients require continuous values), **device** (CPU vs GPU).
- **Broadcasting** lets operations run between tensors of different shapes without explicit loops, improving memory/computation efficiency and enabling GPU parallelism.

- Device management is **explicit** — tensors and models must be moved to the same device (`torch.device("cuda")`); PyTorch never moves them automatically. GPUs matter mainly for training (massive parallel matrix ops); CPUs are often sufficient for inference.

1.2 Autograd and the training loop

- **Autograd**: during the forward pass PyTorch builds a dynamic computational graph (DAG) over tensors with `requires_grad=True`. Calling `.backward()` on the loss computes gradients via reverse-mode automatic differentiation (no manual gradient math needed).
- Three essential training components: **model** (learnable parameters), **loss function** (differentiable, scalar-valued — e.g. MSE for regression, cross-entropy for classification), **optimizer** (updates parameters from gradients, e.g. SGD, Adam).
- Correct training-step order: forward pass → compute loss → `optimizer.zero_grad()` → `loss.backward()` (computes gradients only) → `optimizer.step()` (updates weights only). **Forgetting to zero gradients** causes gradient accumulation across steps — a classic bug that makes loss fluctuate and prevents convergence.
- Gradient descent variants: **Batch** (whole dataset per update), **Stochastic (SGD)** (one point/small batch at a time), **Mini-batch** (small subsets) — mini-batch is the practical default balancing stability and speed.
- Useful tensor utilities: `.item()` (scalar out of a 1-element tensor), `.detach()` (removes a tensor from the graph), `.cpu()`, `.numpy()` (after detach+cpu).
- `nn.Module` is the flexible base class (subclass + define `forward()`); `nn.Sequential` is a simpler container for stacking layers without a custom class.

1.3 Neural network math and activation functions

- Matrix formulation: $\mathbf{Y = XW^T + B}$, where X is (batch N × input features D), W is the weight matrix, B the bias; output is (N × output features M/K). `nn.Linear` stores weights transposed internally but performs the same computation.
- A single artificial neuron: linear combination $z = w^T x + b$ followed by a non-linear activation.
- **Why non-linearity is mandatory**: stacking linear layers without activation functions is mathematically equivalent to ONE linear layer (matrix multiplication is associative) — depth adds no representational power without non-linearity.
- Common activations: **ReLU** (0 for negative, passes positive; risk of "dying ReLU"), **Sigmoid** (0–1, vanishing gradient at extremes), **Tanh** (–1 to 1, zero-centered, still has vanishing-gradient issues at saturation). Output-layer choice depends on task: identity/ linear + MSE for regression; sigmoid (binary) or softmax (multi-class) + cross-entropy for classification.

- **Training pathologies**: divergence (loss exploding — usually learning rate too high), oscillation, plateau. Gradient-norm tracking detects vanishing (near-zero from the start) vs exploding (spikes) gradients. Momentum in optimizers stabilizes learning.
- **Overfitting vs underfitting**: underfitting = low model capacity → high train AND validation error; overfitting = high capacity fitting noise → low train error, high validation error. Capacity grows with depth (layers) and breadth (neurons/layer); breadth increases complexity faster. Early stopping and capacity tuning control overfitting.

Key terms: tensor shape/dtype/device, autograd, computational graph, zero_grad, gradient descent variants, $Y=XW^T+B$, non-linearity necessity, ReLU/Sigmoid/Tanh, vanishing/exploding gradients, overfitting/underfitting, model capacity.

---

2. Convolutional Neural Networks (CNNs)

2.1 Why ANN fails on images, and how convolution fixes it

- Fully-connected (ANN) layers on images cause **parameter explosion**: a 32×32×3 image (3,072 features) connected to 100 neurons needs >300,000 weights; scales up rapidly with image size.
- Flattening an image into a 1D vector for ANN also **destroys spatial structure** — neighboring pixels become distant in the vector, so ANN cannot learn edges/shapes efficiently and tends to overfit.
- Images are represented as **3D tensors**: `(Channels, Height, Width)` — RGB = 3 channels, grayscale = 1 channel; batches use `(N, C, H, W)`.
- **Convolution** is a **linear operation**: at each spatial location it computes the dot product of a flattened image patch with a flattened kernel (w^Tx), summed. It is linear because `T(aX1+bX2) = aT(X1)+bT(X2)`; non-linearity still comes from the activation function applied after, not from convolution itself.
- **Weight sharing**: the same kernel slides across every spatial location (vs. a unique weight per pixel in ANN) — this drastically cuts parameters and assumes patterns can appear anywhere in the image (an *inductive bias*).

2.2 Stride, padding, filters, and parameter formulas

- **Stride** controls how many pixels the kernel shifts per step; larger stride = smaller output, less computation, possible detail loss.
- **Padding** adds zeros around the border to preserve spatial size after convolution.
- Output size formula: `output = (input_size − filter_size + 2·padding) / stride + 1`.
- Multiple filters (kernels) → multiple feature maps stacked as output **depth**. Each filter spans all input channels: shape `[C_in × K × K]`.
- **Parameter count per conv layer**: `(C_in × K × K + 1) × C_out` (the `+1` is the bias per filter). This does **not depend on image height/width** — only on kernel

size and channel counts, which is why CNNs are far more parameter-efficient than fully-connected layers for images (for example, a CNN with ~136K parameters can outperform an ANN with ~789K parameters on the same classification task).
- Parameters for a fully-connected (dense) layer, for comparison: `D_in × D_out + D_out`.

2.3 Pooling and the full CNN pipeline

- **Pooling** (max or average) reduces height/width but **not depth**, and adds **zero trainable parameters**. Max pooling keeps the strongest activation in each window (typically 2×2, stride 2); it introduces translational tolerance (recognizing shifted features).
- Standard CNN block: **Convolution → ReLU → Pooling**, repeated to build depth.
- Full pipeline for classification: input → (conv → ReLU → pool)×N → flatten → fully connected (dense) layers → logits → softmax (multi-class) or sigmoid (binary).
- **Dropout**: randomly zeroes a fraction `p` (commonly 0.2–0.5) of neuron outputs during training only (off at test/inference) to reduce over-reliance on specific neurons and combat overfitting; slows convergence but improves generalization. Implemented as a Bernoulli process; usually applied after flattening / around dense layers, rarely on conv layers in small CNNs.
- **Double descent**: training loss can drop, rise, then drop again as capacity/training progresses — a real, observed phenomenon, not a bug.
- **Inductive biases of CNNs**: **locality** (nearby pixels are related) and **translational invariance** (a pattern can appear anywhere) — these let CNNs generalize with far fewer parameters than ANN.
- **Global Average Pooling (GAP)**: modern alternative to flatten + dense layers at the end of a CNN — averages each feature map to one value, cutting parameters and overfitting risk while keeping performance.

2.4 The ImageNet era, AlexNet, and hierarchical features

- Pre-2012: handcrafted features (SIFT, HOG, Haar) + classical ML, limited by manual feature engineering, small datasets, no GPU support.
- **ImageNet** (1.2M images, 1000 classes) enabled data-driven deep learning; **Top-5 error** is the standard metric — AlexNet cut it from 26% to 15%; EfficientNet later exceeded 93% accuracy.
- **AlexNet innovations**: ReLU activation (faster training, less vanishing gradient), 8 learnable layers (5 conv + 3 FC), two-GPU distributed training, data augmentation (rotate/ flip/color jitter), dropout on FC layers. Input fixed at 227×227×3.
- **Hierarchical feature learning**: early CNN layers detect edges/textures, middle layers detect shapes/parts, deep layers detect whole-object/task-specific concepts.

2.5 Transfer learning

- Training CNNs from scratch needs large labeled data, heavy compute, careful tuning — often impractical. **Transfer learning** reuses features learned on a large dataset (e.g. ImageNet) rather than starting from random weights.

- Two strategies: **Feature extraction** (freeze all pretrained layers, retrain only the final classifier — good for small datasets) vs **Fine-tuning** (also unfreeze some deeper layers, e.g. ResNet's `layer4`, to adapt to the new domain — better accuracy, more compute/time).
- In PyTorch, freeze a layer with `param.requires_grad = False` (stops gradient computation and updates for that layer, speeding up training).
- Guidance: datasets of at least a few thousand images work well with transfer learning; very small datasets (<1,000 images) risk overfitting even with transfer learning; very different domains (e.g. medical imaging) may need training from scratch or specialized methods.
- Preprocessing for pretrained models: resize to the model's expected input (e.g. 224×224) and normalize using the **same statistics the pretrained model was trained on** (e.g. ImageNet mean/std) — critical for transfer learning to work well.
- Popular pretrained backbones: ResNet, VGG, AlexNet, EfficientNet, MobileNet, GoogleNet/ Inception. **ResNet18** typically uses a single FC layer after average pooling; its **skip (residual) connections** mitigate vanishing gradients in deep networks.

2.6 Practical training pipeline and evaluation

- Data pipeline order: load images/labels → organize into `train/val/test` folders (or a DataFrame of paths + labels) with **stratified sampling** to preserve class balance across splits (typical 70/15/15) → resize to fixed size → **augment** (flip, rotate, brightness) — **applied to training data only**, never to validation/test (avoids data leakage, keeps evaluation unbiased) → normalize pixel values → convert to `(C,H,W)` tensors.
- PyTorch's `ImageFolder` auto-labels images from folder names; **`Dataset`** fetches single samples, **`DataLoader`** handles batching/shuffling/iteration.
- **Optimizers**: **Adam** (adaptive learning rates + momentum via `beta1`, `beta2`) generally converges faster and more stably than **SGD** (fixed learning rate, needs careful tuning).
- **Learning-rate scheduler**: reduces LR when validation loss plateaus, for finer optimization. **Early stopping**: halts training if validation loss doesn't improve for a set number of epochs (**patience**), preventing overfitting/wasted compute.
- **Regularization**: **L2** (weight decay) is preferred in PyTorch — smooth, differentiable penalty on large weights, passed as an optimizer parameter. **L1** imposes an absolute penalty, is non-differentiable at zero, and is rarely used directly in PyTorch optimizers.
- **Evaluation beyond accuracy**: the **confusion matrix** (TP/FP/FN/TN) enables **precision** (`TP/(TP+FP)`), **recall** (`TP/(TP+FN)`), and **F1** (harmonic mean of precision & recall). Diagnose fit quality from the train-vs-validation accuracy gap: both low = underfitting; high train / low validation = overfitting; both high with small gap = good fit.

Key terms: parameter explosion, weight sharing, stride/padding formula, pooling, inductive bias, GAP, AlexNet, transfer learning (feature extraction vs fine-tuning), ResNet18 skip connections, Adam vs SGD, L1 vs L2, early stopping, confusion matrix.

---

3. Autoencoders & Generative Models

3.1 Encoder-decoder architecture

- General framework: **Encoder** compresses input into a latent/bottleneck representation; **Decoder** reconstructs an output from it. As a composite function: $\hat{Y} = D(E(X))$.
- Applies broadly beyond NLP: machine translation, image captioning, segmentation, speech recognition, autoencoding, multi-modal AI. For images, CNN encoders + transpose-convolution ("deconvolution") decoders; for 1D data, fully-connected layers.
- **Encoder-decoder vs Autoencoder**: encoder-decoder is general (input/output can differ, usually **supervised**, e.g. segmentation, translation); an **autoencoder** is the special case where input = output, used to reconstruct the input, and is **unsupervised**.
- **U-Net**: encoder (conv + batchnorm + ReLU + max-pool, filters increasing e.g. 16→32→64→128) → bottleneck → decoder (upsampling/transpose-conv + concatenation + conv, filters decreasing). **Skip connections** concatenate encoder feature maps into the matching decoder stage to recover fine-grained spatial detail lost during downsampling — critical for segmentation quality (e.g. medical imaging).
- **Transpose convolution** ("deconvolution") increases spatial dimensions (upsampling), the opposite of standard convolution, which reduces spatial dimensions (downsampling).

3.2 Autoencoder variants

- **Standard autoencoder**: dense or convolutional encoder/decoder trained to minimize reconstruction loss (MSE), e.g. on MNIST. Latent vector size is a hyperparameter.
- **Denoising autoencoder**: trained on **paired noisy/clean data** — encoder extracts informative features while filtering noise, decoder reconstructs the clean image. The latent vector retains only the information needed for the clean reconstruction, discarding noise. Used for satellite-image or MRI denoising.
- **Sparse autoencoder**: imposes sparsity constraints on hidden units for feature extraction.
- **Variational Autoencoder (VAE)**: a probabilistic generative model assuming a Gaussian latent distribution. Encoder outputs mean **μ** and std **σ**; the latent sample uses the **reparameterization trick**: $z = \mu + \sigma \cdot \varepsilon$ where $\varepsilon \sim N(0,1)$, enabling diverse

generation by varying ε while keeping the sampling step differentiable. VAEs tend to produce **blurry** outputs and remain more discriminative than fully generative.

3.3 Generative Adversarial Networks (GANs)

- Motivation: collecting diverse labeled real images is expensive; GANs (Ian Goodfellow, 2014) generate synthetic data via **adversarial learning**.
- **Generative vs discriminative models**: discriminative models (e.g. plain autoencoders) classify/reconstruct observed data; **generative models** learn the underlying data distribution and can produce new, unseen but realistic examples.
- **Architecture**: **Generator (G)** maps random noise (latent vector) → synthetic (fake) data; **Discriminator (D)** classifies inputs as real vs fake. They play a **minimax game**: `min_G max_D E[log D(x)] + E[log(1 − D(G(z)))]` — G tries to fool D, D tries to correctly separate real/fake; ideally converges toward a Nash equilibrium.
- **Training challenges**: **vanishing gradient** (if D becomes too strong early, G gets no useful gradient) and **mode collapse** (G produces limited variety of outputs). Mitigations: loss-function changes, mini-batch training, adjusting G/D update order.
- **Variants**:
  - **Vanilla GAN** — fully-connected layers for both G and D; G outputs a flattened image size (e.g. 784 for 28×28 MNIST) from a latent vector (e.g. size 100); trained with binary cross-entropy, Adam.
  - **DCGAN** — replaces FC layers with convolutions: G uses **transpose convolutions** (upsampling) with batchnorm + **ReLU**; D uses convolutions (downsampling) with batchnorm + **Leaky ReLU**. Leaky ReLU (small negative slope) avoids "dead neuron" issues that plain ReLU can cause in the discriminator.
  - **Conditional GAN (cGAN)** — adds auxiliary label/condition inputs to both G and D, enabling class-controlled generation (e.g. generate only cats).
  - **Wasserstein GAN (WGAN)** — replaces the discriminator's probability output with a Wasserstein-distance scoring function under a Lipschitz constraint, improving gradient quality and training stability (addresses vanishing gradients directly).

Key terms: encoder-decoder vs autoencoder, U-Net skip connections, transpose convolution, denoising autoencoder, VAE reparameterization, GAN minimax game, mode collapse, DCGAN/cGAN/WGAN.

---

4. Object Detection & Image Segmentation

4.1 Foundations: bounding boxes and metrics

- **Object detection** = classify AND localize objects (vs. classification alone).
- **Bounding box formats**: **COCO** (top-left x,y + width,height), **Pascal VOC** (top-left x1,y1 + bottom-right x2,y2), **YOLO** (center x,y + width,height, normalized).
- **IoU (Intersection over Union)** = Area of Overlap / Area of Union, ranges 0–1; measures how well a predicted box matches ground truth.

- From TP/FP/FN/TN: **Precision** = TP/(TP+FP), **Recall** = TP/(TP+FN), **F1** = harmonic mean. **mAP (mean Average Precision)** averages precision across classes and recall levels — the primary accuracy metric for detectors.
- **One-stage vs two-stage detectors**: two-stage (e.g. Faster R-CNN) first generates region proposals, then classifies each; one-stage (YOLO, SSD) directly predicts boxes + classes in a single pass — faster, enabling real-time detection.
- **Classical motion detection** (pre-deep-learning, video): **background subtraction** (threshold pixel differences between a modeled background and current frame) and **optical flow** (tracks pixel motion assuming color constancy + small motion). Both struggle with noise, illumination change, shadows, occlusion — motivating deep-learning approaches.

4.2 YOLO family

- **YOLO v1**: divides image into an S×S grid; each cell predicts a bounding-box vector (object-presence probability, center x/y, width/height, class probabilities — e.g. size 7 for 2 classes); base model has 24 conv + 4 max-pool layers, running at ~45 FPS with ~63% mAP (Pascal VOC 2012), struggles with small/overlapping objects. **Fast YOLO** is a separate, smaller variant (9 conv layers) trading accuracy for speed, reaching ~155 FPS — the ~150 FPS figure belongs to Fast YOLO, not the 24-layer base model.
- **Non-Maximum Suppression (NMS)**: removes redundant overlapping boxes for the same object by keeping the highest-confidence box and suppressing others above an IoU threshold (e.g. 0.3– 0.5). Not used for segmentation.
- **YOLO loss** has 4 components: bounding-box coordinate loss (MSE), object-presence loss, no-object loss (penalizes false detections in background), class-prediction loss.
- **Version evolution**: v2/YOLO9000 (larger input, batch norm, **anchor boxes**; ~76.8% mAP at 416×416 input, ~78.6% mAP at 544×544 input) → v3 (Darknet-53 + residual connections, multi-scale/feature-pyramid detection) → v4 (CSPDarknet backbone, PANet feature fusion, flexible "bag of freebies") → v5 (PyTorch, scalable small/medium/large/xlarge, mosaic/mix-up augmentation) → v6 (edge/embedded deployment, up to ~1242 FPS on small models) → v7 (decoupled heads, focal loss for class imbalance) → v8 (anchor-free heads, C2F layers).
- **YOLOv5 architecture** in three parts: **Backbone** (CSP network, repeated C3 blocks with skip connections, SPPF — multi-scale max pooling + concatenation) extracts low/mid/high-level features; **Neck** (Feature Pyramid Network — upsample, concatenate, C3 blocks without skip connections) fuses multi-scale features; **Head** (uses layers 17/20/23) predicts boxes + classes at multiple scales. Labels: class ID + normalized center x/y/width/height (YOLO format), configured via a `.yaml` file; trained via transfer learning from pretrained weights (e.g. `yolov5s.pt`).

- **SSD (Single Shot MultiBox Detector)**: one-stage like YOLO; applies multiple feature layers at different scales with multiple default anchor boxes for multi-scale detection.

4.3 Image segmentation

- Types: **Instance segmentation** (distinguishes individual objects of the same class), **Binary segmentation** (foreground vs background), **Multi-class segmentation** (pixel → one of several classes).
- **U-Net** (recap): encoder-decoder with skip connections, precise localization.
- **U-Net++**: adds **dense skip connections** across multiple scales + intermediate upsampling to reduce the semantic gap between encoder/decoder features — more precise than plain U-Net.
- **Residual U-Net**: adds residual connections inside conv blocks, easing training of deeper segmentation networks and mitigating vanishing gradients (useful for challenging domains like underwater imaging with low light/varying shapes).
- Challenges: precise boundary localization, variable object scale/overlap, illumination/ shadow, complex backgrounds; medical segmentation (e.g. tumors) must avoid both under- and over-segmentation.

4.4 Classical (pre-deep-learning) detectors: LBP, HOG, Haar

- **Local Binary Patterns (LBP)** — texture descriptor: for each pixel, compare it to its 3×3 neighbors (1 if neighbor < center else 0), forming an 8-bit binary code → decimal value; a histogram of these codes over a region is the feature vector, used for face detection.
- **Histogram of Oriented Gradients (HOG)** — shape/edge descriptor: resize image → compute x/y gradients → gradient **magnitude** (`√(gx²+gy²)`) and **direction** (`atan2(gy,gx)`) → bin directions into per-cell histograms (e.g. 8×8-pixel cells) → normalize over blocks of adjacent cells (illumination invariance) → concatenate into a feature vector. Several binning methods exist (0–180° with many bins, coarser 9-bin grouping, magnitude-weighted, interpolated bins). Used with classifiers like SVM for detection (e.g. pedestrians).
- **Haar features / Haar cascades** — intensity-difference features (edge/line/ rectangle/ correlation types) over black/white rectangular regions; computed efficiently via the **integral image** (cumulative pixel-sum table). Weak classifiers combined via **AdaBoost** into a **cascade** of stages, where early stages quickly reject non-object regions — very fast for face/eye detection (OpenCV `CascadeClassifier` + `detectMultiScale`).
- **Hybrid pipeline**: classical detectors (Haar/HOG) can crop candidate regions which are then passed to a deep-learning model (ResNet/VGG) for more robust final classification.

Key terms: IoU, mAP, NMS, YOLO loss (4 parts), YOLO v1–v8 evolution, YOLOv5 Backbone/Neck/Head, U-Net vs U-Net++ vs Residual U-Net, LBP, HOG, Haar cascade + AdaBoost + integral image.

---

5. Recurrent Neural Networks: RNN, LSTM, GRU

5.1 Why sequences need memory

- CNNs assume fixed-size, order-independent grid data — unsuitable for **sequential data** (text, speech, time series, sensor data, video) where order matters and length varies.
- **Sequence task taxonomy**: **Many-to-one** (sequence → single label, e.g. sentiment classification), **One-to-many** (single input → sequence, e.g. image captioning), **Many-to-many** (sequence → sequence, e.g. translation, next-token prediction).
- Pre-RNN approaches used **padding/truncation** to force fixed length — causes information loss or wasted computation. **Tokenization** converts text into a numeric vocabulary index sequence so neural nets can process it; word order in tokens must be preserved (order changes meaning).

5.2 Vanilla RNN

- **Hidden-state update**: `h_t = tanh(W_x·x_t + W_h·h_{t−1} + b)`; output `y_t = W_y·h_t + b_y`. Tensor shapes: `x_t: (B,D)`, `h_t: (B,H)`, `y_t: (B,O)`.
- **Recursion analogy**: like Fibonacci/factorial, `h_t = f(x_t, h_{t−1})` depends on the prior step, with a base condition `h_0` (usually zero).
- **Weight sharing across time steps**: the same `W_x, W_h, W_y` are reused at every step — drastically fewer parameters than one network per position, and generalizes to variable sequence length.
- **tanh over ReLU**: tanh is zero-centered, bounded (–1 to 1), stabilizing hidden-state values; ReLU/Leaky ReLU are not zero-centered and can cause exploding values in a recurrent loop.
- **Backpropagation Through Time (BPTT)**: total loss `L = Σ L_t`; gradients flow backward through every time step, updating the shared weights.
- **Many-to-one vs many-to-many training**: many-to-one computes loss once (last hidden state only); many-to-many computes loss at every step.
- **Core limitation**: the hidden state has a **fixed size** and must compress all sequence history — as sequences get longer, **earlier information fades** (e.g. “the movie was boring at first but became amazing later” can be misclassified because recent tokens dominate). Root cause: **vanishing gradients** — repeated multiplication of small derivatives (tanh saturates near ±1) shrinks gradients exponentially over long sequences; **exploding gradients** occur if the recurrent weight matrix norm exceeds 1. Vanishing is more common and more damaging.

- The same input token can produce different output depending on the accumulated hidden-state context at that position — this context-sensitivity is central to why RNNs work at all.

5.3 LSTM and GRU: gated solutions

- **LSTM (Long Short-Term Memory)** separates memory into: **cell state (C_t)** — long-term memory, updated **additively**; **hidden state (h_t)** — short-term, exposed output. Three gates (all **sigmoid**, scaling 0–1):
  - **Forget gate** — how much old cell-state to keep.
  - **Input gate** — how much new candidate information to write.
  - **Output gate** — how much of the cell state to expose as the hidden state.
  - Candidate memory uses **tanh**. The **additive** cell-state update path (forget-gate × previous cell + input-gate × candidate) is what preserves gradient flow better than the vanilla RNN's full-replacement update — conceptually similar to ResNet skip connections.
  - LSTMs are heavier computationally, less suited to edge/mobile deployment.
- **GRU (Gated Recurrent Unit)**: simpler — **one** hidden state (no separate cell state), two gates: **update gate (z_t)** (balances old vs new — close to 1 keeps more past, close to 0 favors new) and **reset gate (r_t)** (controls how much past hidden state feeds the candidate state). Fewer parameters, faster to train, often competitive with LSTM. Gate math uses **element-wise (Hadamard) products**, not dot products.
- No universal winner between RNN/LSTM/GRU — choice depends on sequence length, dataset size, compute budget.

Aspect	TF-IDF	Vanilla RNN	LSTM
Representation	Static, sparse	Dynamic hidden state	Dynamic cell + hidden state
Captures order?	No	Yes	Yes (with gates)
Long-term dependency	No	Limited (vanishing gradients)	Yes (gated, additive)
Memory structure	None	Single hidden state	Separate cell + hidden state

5.4 Character-level text generation and decoding

- **Character-level LSTM** predicts the next character given a sequence: build vocab of unique characters → char-to-index / index-to-char maps → input sequence → shifted target (e.g. input "hello" → target "ello"). Pipeline: **Embedding** (B×T×E) → **LSTM** (B×T×H) → **Linear** to vocab logits (B×T×V); loss = cross-entropy per step (requires flattening logits/targets to 2D).
- **Decoding strategies**: **Greedy decoding** always picks the highest-probability character — simple but repetitive/deterministic. **Temperature-scaled sampling**

divides logits by a temperature **τ** before softmax: **low τ (<1)** sharpens the distribution (more deterministic/safe), **high τ (>1)** flattens it (more diverse/creative, riskier). Temperature is applied **at inference only**, not training — it is a bias/variance dial for generation.
- **Auto-regressive generation**: the predicted token is fed back as the next input.

5.5 Why recurrence still isn't enough
- Even with gating, RNN/LSTM/GRU process tokens **strictly sequentially** — no parallelism — and information for very distant dependencies must still pass through many intermediate steps (long information paths), with the hidden size acting as a bottleneck. This motivates **attention mechanisms and Transformers** (Section 7).

Key terms: hidden-state recursion, weight sharing across time, BPTT, vanishing/exploding gradients, LSTM cell vs hidden state, 3 gates, GRU update/reset gates, temperature sampling, sequential bottleneck.

---

6. NLP Foundations & Word Embeddings

6.1 Text preprocessing pipeline
- **NLP** = computational processing of text; text must be converted to vectors before any neural network can use it.
- Standard pipeline: **Tokenization** (split into words/subwords — naive `.split()` fails on contractions like "don't"; NLTK's `word_tokenize` handles this correctly) → **lowercasing** → **stop-word removal** (removes low-information words like "is", "the") → **stemming** (crude rule-based suffix chopping, fast but can produce non-words, e.g. "amaz") vs **lemmatization** (uses vocabulary/morphology for valid roots, more accurate, benefits from **POS-tagging** first).
- **Task dependence is critical**: preprocessing is not a fixed checklist. For **sentiment analysis**, stop-word removal must **retain negation words** ("not", "never") — removing them flips meaning (e.g. "not good" → "good"). Numbers may carry meaning (e.g. star ratings) and shouldn't always be stripped.

6.2 From counting to dense vectors
- **One-hot encoding**: each word → sparse vector, one index = 1. Avoids false numeric bias (e.g. "2 > 1" implying "queen > king") but cannot capture semantic similarity and scales vector size to vocabulary size (curse of dimensionality).
- **Bag of Words (BoW)**: raw word-frequency counts, ignores order/grammar; sparse, high-dimensional.
- **TF-IDF (Term Frequency–Inverse Document Frequency)**: `TF` = frequency within a document (normalized); `IDF` = inverse of frequency across documents (log scale) — down-weights common words, up-weights distinctive ones. Still ignores word order (bag-of-words blind spot); still sparse. Practical rule: fit the vectorizer (`fit_transform`)

only on training data, then `transform` (not `fit_transform`) the test data, to avoid data leakage.

- **Co-occurrence matrix**: counts how often word pairs appear together within a window across the corpus (symmetric matrix); captures broader/global context than a single sliding window, but grows as vocab² — needs dimensionality reduction (→ leads to GloVe, Section 8).
- **Word2Vec** (prediction-based): neural embeddings trained via **Skip-gram** (target word → predict context words; better for large datasets) or **CBOW** (context words → predict target; better for smaller datasets, ignores context word order/treats it as a "bag"). Captures semantic **vector arithmetic**: `king − man + woman ≈ queen`, `Paris − France + Italy ≈ Rome`. Trained via **negative sampling** (binary classification of true vs random word pairs — cheaper) or **hierarchical softmax** (binary-tree-structured vocabulary) as alternatives to full softmax, which is expensive over large vocabularies.
  - **Weighted CBOW**: replaces simple averaging of context embeddings with a **weighted sum** (e.g. by distance/attention/frequency) for better prediction accuracy.
  - Implementation notes: embedding layer + linear layers + softmax, **NLLLoss**, SGD (lr≈0.01), trained for roughly 100 epochs in typical setups.
- Dense embeddings vs one-hot: much lower dimensional, capture similarity via **cosine similarity** (angle) — words in similar contexts (e.g. "coffee"/"tea") end up close in vector space. Limitation of *static* embeddings: one fixed vector per word regardless of context (the "bank" — river vs finance — ambiguity, i.e. **polysemy**), motivating contextual embeddings (Section 8).
- **Encoding vs Embedding**: encoding is any transform/compression of data; embedding specifically targets dense vectors that capture semantic/syntactic relationships — embedding is a *kind* of encoding.
- **Word-embedding taxonomy**: discrete (one-hot) → frequency-based (BoW, TF-IDF, co-occurrence) → prediction-based (Word2Vec) → global-statistics (GloVe) → contextual (ELMo, BERT, GPT).
- **TSNE vs PCA** for visualizing high-dimensional embeddings in 2D/3D: TSNE is non-linear and better preserves local structure (controlled by a "perplexity" parameter); PCA is linear.

6.3 Classical ML for text classification

- **TF-IDF + classical ML** (Naive Bayes, SVM) remains a strong baseline for **sentence/document-level** classification (e.g. sentiment, spam) — Word2Vec gives only word-level vectors, and naively averaging them loses word order/negation, making TF-IDF often more suitable for classical classifiers.
- Practical workflow: negation-aware preprocessing → TF-IDF vectorization (`fit_transform` train / `transform` test) → train Naive Bayes / Linear SVM (stratified

80/20 split) → evaluate with accuracy, confusion matrix, classification report (precision/recall per class).

- **Naive Bayes** is simple/fast; **SVM** handles high-dimensional sparse data well. Neither handles sarcasm or complex semantics well — deep learning is better suited there.

Key terms: tokenization vs vectorization, stemming vs lemmatization, task-dependent preprocessing, one-hot vs dense embeddings, BoW, TF-IDF (fit_transform vs transform), co-occurrence matrix, Word2Vec (CBOW/Skip-gram), negative sampling, polysemy, TSNE vs PCA.

---

7. Attention Mechanism & Transformer Architecture

7.1 Why attention: the bottleneck problem

- Recurrent encoder-decoders compress an entire source sequence into **one fixed-size vector** (like summarizing a 300-page book on one page) — a severe bottleneck for long sequences, losing fine-grained detail needed for tasks like translation.
- **Attention** lets each output step dynamically look back at **all** encoder states rather than relying on one compressed vector — an "open-book exam" instead of a "closed-book" recap.
- **Soft vs hard selection**: hard selection picks exactly one token (non-differentiable, hard to train); **soft selection** assigns a weight to every token via **softmax**, keeping the whole mechanism differentiable and trainable by gradient descent.

7.2 Encoder–decoder (cross) attention

- Instead of only the final hidden state, keep **all** encoder hidden states in a matrix **H** $\in$ $\mathbb{R}$**^(T×D_H)** (T = source length). Decoder state $\mathbf{S}_t$ (query) is compared against every row of H (keys) to get **alignment scores**.
- **Dot-product attention**: `E_T = H·S_T` (simple, no learnable parameters, fast, assumes linear similarity).
- **Additive attention**: projects decoder/encoder states via learnable `W_s`, `W_h`, applies `tanh`, then a learnable vector `v_a` reduces to a scalar score — more expressive/non-linear, more compute/memory.
- Raw scores are **not** probabilities — apply **softmax** to normalize into attention weights (summing to 1); for numerical stability, subtract the max score before exponentiating.
- **Context vector**: `C_T = α_T^T · H` — a weighted sum of encoder states (not a simple average).
- Decoder prediction: concatenate `S_t` and `C_t` → linear `+ tanh` → project to vocabulary logits via `W_0` → softmax → predicted token.
- Across multiple decoder steps, attention forms an **S×T attention matrix** (rows = target steps, columns = source tokens); each row sums to 1 (softmax); a strong

diagonal pattern suggests monotonic (word-order-matching) alignment between source and target languages.

- **Cross-attention** (query from decoder, key/value from encoder) connects two *different* sequences; **self-attention** (query, key, value all from the *same* sequence) lets tokens contextualize themselves — this is the core of Transformers.

7.3 Self-attention and scaled dot-product attention

- Input embeddings form matrix **X** $\in \mathbb{R}$**^(N×D_model)** (N tokens); initially static/independent.
- Self-attention projects X via three learnable weight matrices: `Q = XW_Q`, `K = XW_K`, `V = XW_V` (shapes `N×D_k`). Query = what a token is looking for; Key = what a token offers; Value = the content it carries forward.
- **Attention score matrix**: $S = QK^T \in \mathbb{R}^{(N \times N)}$ — unnormalized, unbounded.
- **Scaling**: divide by $\sqrt{d_k}$ before softmax — as `d_k` grows, raw dot products grow in magnitude (variance grows ~`d_k`), which makes softmax overly **sharp** (near one-hot), starving most tokens of gradient (vanishing gradients for low-attention tokens). Scaling keeps the distribution smoother and training stable.
- **Full formula**: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$. Output $Z \in \mathbb{R}^{(N \times d_k)}$ — same dimension as V, but now **context-aware** (each token's representation is informed by every other token, weighted by relevance).
- **Self- vs cross-attention** recap: self = Q,K,V all from one sequence; cross = Q from decoder, K/V from encoder.

7.4 Multi-head attention and the encoder block

- A single attention head captures only **one** type of relationship pattern (analogous to one CNN filter). **Multi-head attention** runs `H` heads in parallel, each with its **own** `W_Q^r, W_K^r, W_V^r` projections (dimension `d_k = d_model / H` per head, so concatenation restores `d_model`) — different heads can specialize in different relationships (e.g. negation vs syntax).
- Steps: for each head, compute Q/K/V → scaled dot-product attention → output `(N×d_k)` per head → **concatenate** all heads → `(N×d_model)` → multiply by output projection **W_O** (learned mixing across heads) → final MHA output `(N×d_model)`.
- Requirements: `d_model` must be divisible by number of heads; MHA output shape must equal input shape (needed for the residual/skip connection).
- **Transformer encoder block** = MHA + residual + LayerNorm + FFN + residual + LayerNorm:
  1. `X1 = LayerNorm(X + MHA(X))` — "Add & Norm".
  2. `X2 = LayerNorm(X1 + FFN(X1))` — second "Add & Norm".
  - **Residual (skip) connections** are **element-wise addition** (not concatenation); preserve original token information and improve gradient flow, enabling deep stacks.

- **Layer normalization** normalizes each token vector independently across the feature dimension: `Z_norm = (Z − μ)/√(σ²+ε)`, then applies learnable scale **γ** and shift **β** (shape = `D_model`). Different from **batch normalization** (normalizes across the batch — suited to CNNs, not sequence models where token count/order varies).
- **Pre-LN vs Post-LN**: the original paper applies LayerNorm *after* the residual add (Post-LN); many modern implementations apply it *before* the sublayer (**Pre-LN**), preferred for better gradient flow/training stability in deep stacks.
- **Position-wise Feed-Forward Network (FFN)**: applied **independently per token** (no token mixing — mixing happens only in MHA); two linear layers with non-linearity between them, expanding `d_model → d_ff` then compressing back `d_ff → d_model` (e.g. 256⟶1024⟶256). Original paper used **ReLU**; many modern models (BERT) use **GELU** (smoother, allows some negative values through, differentiable). FFN parameters: layer 1 = `d_model·d_ff + d_ff`, layer 2 = `d_ff·d_model + d_model` — FFN is typically the majority of a Transformer's total parameters. Same FFN weights are **shared across all token positions** (a kernel-like reuse, analogous to CNN weight sharing).
- **Masking**: used in **decoder** self-attention to block attention to future tokens (autoregressive generation) and to ignore padding; **not** used in bidirectional encoders.

- **Positional encoding**: since attention has no inherent notion of order, position must be injected — classic sinusoidal formula: `PE(pos,2i) = sin(pos/10000^(2i/d_model))`, `PE(pos,2i+1) = cos(pos/10000^(2i/d_model))`.
- Stacking multiple encoder blocks refines the representation (not sequence length) — earlier layers tend to capture local relations, later layers more abstract meaning. A **single** encoder block alone is **not** a complete NLP model — it's missing tokenization, vocabulary, embeddings, pooling, and a classifier head.
- To build an actual classifier: raw text ⟶ tokenizer + vocabulary ⟶ token embedding + positional embedding ⟶ encoder block(s) (with an **attention mask** distinguishing real tokens `1` from padding `0`) ⟶ pooling (mean/max, token-level ⟶ sentence-level) ⟶ linear classification head ⟶ logits.

Key terms: QKV, soft vs hard attention, dot-product vs additive attention, scaled dot-product `softmax(QKᵀ/√d_k)·V`, multi-head attention (per-head projections + W_O), residual connection, layer norm vs batch norm, Pre-LN vs Post-LN, position-wise FFN, positional encoding, attention mask, pooling + classifier head.

---

8. BERT, GPT, ELMo, GloVe & the LLM Landscape

8.1 GloVe and ELMo: bridging word2vec and contextual models

- **GloVe (Global Vectors)**: unsupervised, builds a **global co-occurrence matrix** across the whole corpus (not just a local sliding window as in Word2Vec), converts counts to probabilities, and derives embeddings preserving these global statistics (e.g. “ice” close to “cold”/“solid”, “steam” close to “hot”/“gas”). Window size still shapes what counts as “context.”
- **Polysemy problem**: any *static* embedding (Word2Vec, GloVe) assigns **one vector per word**, ignoring that words like “bank” mean different things in different contexts — reduces downstream accuracy.
- **ELMo (Embeddings from Language Models)**: generates **contextual** embeddings via a **character-level CNN** (captures morphology — prefixes/suffixes/spelling, helps with rare/ OOV words) refined through highway networks, then a **bidirectional LSTM** (left-to-right + right-to-left) — output is 1024-dim = 512 forward + 512 backward features, differing per sentence context. Downsides: computationally expensive (sequential LSTM), can still suffer vanishing gradients on long sequences, less efficient than Transformers.

8.2 BERT: Bidirectional Encoder Representations from Transformers

- Uses the Transformer **encoder** only, bidirectionally (context from both left and right simultaneously) — Google AI, 2018.
- **Three embeddings summed** at input: **Token** embedding, **Segment** embedding (distinguishes sentence A/B via `[CLS]`/`[SEP]`), **Positional** embedding (sinusoidal).
- **Pretraining objectives**:
  - **Masked Language Modeling (MLM)** — randomly masks 15% of tokens, model predicts them from surrounding (bidirectional) context.
  - **Next Sentence Prediction (NSP)** — predicts whether sentence B logically follows sentence A (vs a random pairing).
- **Fine-tuning** adapts pretrained BERT to a downstream task (e.g. Q&A, sentiment) using labeled data.
- Specs: **BERT-base** = 12 layers, 768 hidden size, 12 attention heads, ~110M parameters; **BERT-large** = 24 layers, 1024 hidden, 16 heads, ~340M parameters (trained on TPUs).
- **Variants**:
  - **DistilBERT** — smaller/faster (6 layers) via **knowledge distillation** (student model learns from a larger teacher model), similar accuracy at lower cost.
  - **RoBERTa** — same architecture, improved *training strategy*: more data, longer training, dynamic masking instead of NSP, larger batches.
  - **ALBERT** — reduces parameters via **cross-layer weight sharing** and **factorized embedding parameters**.

- **DeBERTa — disentangled attention** separating content and positional information for better dependency modeling.

8.3 GPT: Generative Pre-trained Transformer

- Uses the Transformer **decoder** only, with **masked** multi-head self-attention — **auto-regressive**: predicts the next token from prior tokens only (unidirectional), never sees the future during self-attention.
- Tokenization: **Byte Pair Encoding (BPE)** — subword units handle unknown words and control vocabulary size.
- Pipeline: tokenization → embedding → unsupervised pretraining (next-token prediction) → supervised fine-tuning → **RLHF** (Reinforcement Learning from Human Feedback — humans rank outputs to align responses, reduce rude/incorrect answers) → inference (auto-regressive generation).
- Evolution: GPT-1 (117M params) → GPT-2 (1.5B) → GPT-3 (175B, supports few-shot learning) → GPT-3.5/GPT-4.5 (parameter count undisclosed by OpenAI, widely estimated in the trillions; multi-modal — text/image input — better reasoning/safety).

8.4 BERT vs GPT, and encoder/decoder-only implications

	BERT	**GPT**
Transformer part	Encoder only	Decoder only
Direction	Bidirectional	Unidirectional (autoregressive)
Pretraining	MLM + NSP	Next-token prediction
Typical use	Understanding (classification, extractive Q&A)	Generation
Developer	Google AI	OpenAI

- **Encoder-only models don't "hallucinate"** in the generative sense — they select/extract from given input rather than generate free text; **decoder-only (generative) models** can produce fluent but unsupported ("hallucinated") text.
- Shared challenges: compute cost, limited context-window length, possible hallucination/bias, reasoning limited by training-data quality/diversity.

8.5 Graph Neural Networks (GNN)

- Motivation: CNNs assume grid-structured data, RNNs assume sequential order — **GNNs** handle arbitrary **graph-structured data** (nodes with varying, non-grid connectivity), e.g. social networks, molecules, knowledge graphs.
- **Nodes** (entities, with feature vectors) and **edges** (relationships, possibly weighted/directed) form a graph; represented mathematically by an **adjacency matrix** (N×N, entries = edge weight or 0/1 presence).

- **Message passing** = the core operation: **aggregate** neighbor features (sum/ mean/max/ attention) → **update** the node's own feature by combining with the aggregated message (residual-like). Stacking layers reaches **k-hop neighbors** (1-hop = direct neighbors, 2-hop = neighbors-of-neighbors); typically only 2–3 layers are used (deeper → more compute, risk of over-smoothing/vanishing gradients) — a receptive-field vs cost trade-off.
- Full pipeline: input graph → message passing (several layers) → node embeddings → **graph pooling** (sum/mean/max to get one graph-level vector) → MLP → task output.
- **Variants**: **GCN (Graph Convolutional Network)** — applies a learnable weight matrix to aggregated neighbor features, normalized by adjacency/node-degree; **GAT (Graph Attention Network)** — learnable **attention weights** per neighbor (focus on more relevant neighbors); **MPNN (Message Passing Neural Network)** — extends learnable weighting to edge features too.
- Applications: social-network/friend recommendation, molecular property prediction, fraud detection (transaction networks), traffic forecasting, knowledge-graph reasoning.

Key terms: polysemy, ELMo (char-CNN + biLSTM), BERT (MLM+NSP, 3 embeddings, CLS/SEP), DistilBERT/RoBERTa/ALBERT/DeBERTa, GPT (decoder-only, BPE, RLHF), encoder-only vs decoder-only, GNN message passing, k-hop neighborhood, GCN vs GAT vs MPNN, adjacency matrix.

---

9. Building & Fine-Tuning Transformers in Practice (Hugging Face)

9.1 From-scratch implementation lessons

- Building a transformer encoder block from scratch (PyTorch) confirms: attention-row weights always sum to 1 (softmax); output token shape always equals input shape (needed for residual stacking); a small toy encoder block can have as few as ~2,000 parameters, with the FFN contributing the most.
- Stacking 2–3 encoder blocks (`nn.ModuleList`) refines representation through repeated attention + FFN cycles — **not** a change to sequence length. On tiny/toy datasets, deeper stacks can **overfit worse** than a single block (more capacity, same limited data) — representation quality on realistic data matters more than depth alone.
- Visualizing hidden states/attention maps layer-by-layer shows earlier layers often capture local token relations, later layers more abstract/phrase-level meaning.

9.2 Hugging Face pipelines

- `pipeline("sentiment-analysis")` etc. hides tokenization, attention masking, forward pass, and softmax behind one call — convenient, but does **not** allow fine-tuning.

- `AutoTokenizer + AutoModel...` classes give the transparency needed to inspect/ modify the pipeline and to fine-tune.
- **Batch tokenization**: real applications tokenize batches of variable-length sentences — the tokenizer auto-pads/truncates to a uniform tensor shape and returns an **attention mask**. Batched output logits have shape `[N, C]` (N sentences × C classes).
- **Logits → probabilities**: raw model outputs (logits) are not directly interpretable; apply **softmax** to get class probabilities.

9.3 Extractive Question Answering

- Encoder-only Q&A models output **start_logits** and **end_logits** per token (shape `[1, seq_len]`); answer span = token with max start score to token with max end score.
- **Pitfall**: choosing `argmax` on start and end **independently** can produce an invalid span (start after end) — fix with a helper that searches valid `(start ≤ end)` pairs within a reasonable span length (an **O(n²)** brute-force search over sequence length `n`).
- **Confidence score**: typically the mean of start and end probabilities.
- Encoder-only Q&A is strictly **extractive** — it can only return text present in the given context; if the answer isn't in context, expect low-confidence or irrelevant output. Better/ more relevant context → more accurate answers.
- Long documents exceed the model's max input length (often 384–512 tokens) — requires **chunking** (see Section 10).

9.4 Fine-tuning considerations

- Fine-tuning extractive Q&A needs labeled **(question, context, answer text, answer start position)** tuples — labor-intensive to prepare.
- Fine-tuning is warranted for domain-specific knowledge not in pretrained models (medical, legal) or when data can't leave your infrastructure (privacy) — **not** the default first fix for a weak system (see Section 10.7).
- Standard overfitting mitigations apply during fine-tuning: dropout, parameter reduction, L1/L2 regularization.
- **Encoder-only vs decoder-only vs encoder-decoder** for generation tasks: encoder-only = pure understanding (Q&A extraction, classification); decoder-only = pure generation (GPT-style, auto-regressive); **encoder-decoder** (e.g. T5) combines reading context with generating new text — well suited to grounded generative Q&A (RAG's generative half, see Section 10).

Key terms: pipeline abstraction, batch tokenization + attention mask, start/end logits, invalid-span pitfall, extractive Q&A limitations, when fine-tuning is warranted.

---

10. Retrieval-Augmented Generation (RAG) Systems

10.1 Document ingestion: extraction, cleaning, chunking

- **PDF extraction** (e.g. via `PyMuPDF`/`fitz`) works well on text-based PDFs; scanned/image PDFs need OCR. Common issues: two-column layouts, tables, equations, headers/footers, broken words across lines, invisible Unicode characters.
- **Text cleaning**: whitespace normalization, removing tokenizer-special-tokens and invisible characters, fixing broken words — validated by comparing raw vs cleaned word counts (avoid over-cleaning that removes real content).
- **Chunking** is required because transformers have a max input length (~384–512 tokens). Strategies, from simplest to most advanced:
  - **Fixed-size (word-count) chunking** — simple, but can cut sentences mid-way.
  - **Sentence-aware chunking** — groups whole sentences up to a word budget, with 1–2 sentence overlap between chunks to preserve continuity across boundaries — generally better retrieval quality than fixed-size.
  - **Section-aware chunking** — respects document structure (abstract/intro/methods/results).
  - **Semantic chunking** — detects topic shifts via embedding similarity between adjacent sentences/paragraphs and splits there.
  - **Token-aware chunking** — sizes chunks by actual token count (not words), important since token length varies with the tokenizer and the paired question length.
  - Each chunk retains **metadata**: chunk ID, page number, word/char counts — essential for traceability and debugging.

10.2 Retriever-reader architecture

- **Brute-force Q&A** (run the reader on every chunk) is `O(N)` in chunk count — slow, and most chunks are irrelevant, sometimes causing misleading answers.
- **Retriever-reader split**: **Retriever** quickly finds the top-**K** relevant chunks; **Reader** (extractive Q&A model) processes only those `K` chunks. Cuts reader calls from `N` to `K` (`K` $\ll$ `N`) — complexity becomes `T_retrieve + K·T_reader`.
- **Text embeddings + cosine similarity**: text embeddings (e.g. Sentence Transformers, often 384-dim) place semantically similar text close together in vector space; **cosine similarity** (angle between vectors) — not exact keyword match — finds relevant chunks even when wording differs (e.g. “limitations” vs “disadvantages”). Normalizing embeddings to unit vectors makes cosine similarity equal to a plain **inner (dot) product**.
- **FAISS** (Facebook AI Similarity Search): builds an index over chunk embeddings for efficient nearest-neighbor search (better than linear `O(N)` scanning). `IndexFlatIP` (inner product) on normalized embeddings implements cosine similarity search exactly; approximate indexes (**IVF**, **HNSW**) trade some precision for much greater

speed at large scale. Distance conventions: for **cosine similarity**, higher = more similar; for **L2 distance**, lower = more similar.

- **Combining retriever + reader scores**: normalize both (min-max, 0–1 range) then **multiply** (not add) — both retriever relevance AND reader confidence must be high for a trustworthy final answer (joint-probability-style logic).

10.3 Extractive vs generative Q&A, and RAG proper

- **Extractive Q&A** (BERT-style): returns a text span already present in context — low hallucination risk, but can't synthesize or explain, and fails if the exact answer text isn't present.
- **Generative Q&A / RAG**: an **encoder-decoder** generative model (e.g. T5, Flan-T5) produces new natural-language text grounded in retrieved context — better for explanations, summaries, comparisons; risk of **hallucination** if context is weak or the prompt allows it.
- **Suitability rule of thumb**: extractive is better for short, exact factual answers (acronyms, dates, numeric values, names); generative is better for explanatory/ summary/ conversational answers. A production system often needs **both**, routed by question type.

10.4 Building the RAG prompt

- Formal framing: `Q` = user question, `RQ` = retrieved chunks, `P` = constructed prompt (instruction + retrieved context + question), `G` = generative model, `A` = generated answer.
- **Prompt components**: an explicit instruction to answer *only* from provided context (with a defined fallback for insufficient context), the retrieved context itself, and the user's question.
- **Token-aware, not character-aware, prompt limits**: models truncate by *tokens*, so budgeting by character count risks silent truncation; keep total prompt tokens within a safe limit (e.g. 512), reserving room for the question and instructions.
- **Never feed chunk metadata (page numbers, chunk IDs) into the generation prompt** — the model may copy or hallucinate around it; display metadata **separately** to the user for traceability instead.
- Prompt length isn't "more is better" — irrelevant or excessive context can confuse a small generative model; both prompt length and top-K chunk count are hyperparameters to tune.

10.5 Debugging RAG pipelines

- Two failure classes: **retrieval failure** (correct chunk never retrieved — poor chunking, weak embeddings, phrasing mismatch, too-strict threshold, noisy data) vs **reader/generation failure** (right chunk retrieved, wrong answer extracted/ generated — implicit questions, answer spread across sentences, weak instruction-following in small generative models).

- **Always verify retrieval before blaming the reader/generator** — inspect retrieved chunks manually first.
- Use structured debug outputs (e.g. data frames with chunk ID, page, score, preview) rather than raw print statements; keyword-presence checks in retrieved chunks are a cheap heuristic for likely retrieval failure.
- **Grounding / support categories** for generated answers: fully supported, partially supported, weakly supported, insufficient context — displayed alongside the answer to build user trust and catch unsupported claims.
- **Source-context verification is essential in high-stakes domains** (legal, medical, academic) — always show the supporting context so a human can check correctness; models can produce plausible-but-wrong spans/text.

10.6 Hybrid retrieval and query routing

- Pure semantic (embedding) retrieval can miss exact-match needs (abbreviations, metric names, numbers) — **hybrid retrieval** combines semantic similarity with **keyword-based** methods (e.g. BM25) for more precise results on such queries.
- **Query routing**: classify the incoming question by type (factual / explanatory / summary) and route it to the extractive pipeline (factual) or the generative RAG pipeline (explanatory/summary) — can start rule-based (keyword matching) and later use a trained classifier or an LLM-based router.

10.7 When to fine-tune (and what to fix first)

- Fine-tuning should **not** be the first response to a weak RAG system. Fix, in order:
  1. PDF extraction & cleaning quality.
  2. Chunking & retrieval (including moving to hybrid retrieval).
  3. Query routing (extractive vs generative).
  4. Prompt design & grounding checks.
- Fine-tune only when: domain adaptation/style consistency is needed, the query router needs better accuracy, retrieval ranking needs improvement, or generator output format needs configuring. Requires curated, diverse labeled data (factual/ explanatory/summary/comparison/ unanswerable/refusal-with-citation examples) and is resource-intensive (GPU-bound).
- Deployment/production notes (introduced briefly): containerize with Docker, host on cloud (AWS/Azure), expose via REST APIs (FastAPI/Flask), UI via Streamlit; version pinning and reproducible environments matter for production reliability.

Key terms: sentence-aware/section-aware/semantic/token-aware chunking, retriever-reader architecture, cosine similarity, FAISS (`IndexFlatIP`, IVF/HNSW), extractive vs generative Q&A, RAG prompt (Q/RQ/P/G/A), token-aware prompting, grounding/support categories, hybrid retrieval (BM25), query routing, fine-tune-last principle.

---

11. Other Topics Briefly Introduced (context, not deeply taught)

These were mentioned as forward pointers, tooling notes, or discussion asides rather than taught in depth — useful context, unlikely to be tested in detail:

- **Agentic AI / orchestration frameworks**: LangChain, LangGraph were named as tools for building agentic AI systems on top of LLMs and RAG pipelines.
- **Deployment & MLOps**: containerization (Docker, Kubernetes), cloud hosting (AWS, Azure), REST APIs (FastAPI, Flask, Node.js), frontend hosting (Vercel) — flagged as essential complementary skills, not covered hands-on.
- **Evaluation limitations**: BLEU and similar automatic metrics were noted as having significant limitations for open-ended NLP generation; human-in-the-loop validation remains important.
- **AI ethics, hallucination, and sustainability**: brief reflective discussion on model hallucination risk, keeping humans in the loop for high-stakes decisions, and the energy/ water footprint of large-scale AI data centers.
- **External learning resources**: 3Blue1Brown videos (visual intuition for attention/ transformers), BeautifulSoup/regex for web scraping, general encouragement to read the original “Attention Is All You Need” (2017) paper.